TECHNICAL REPORT

Multi-Objective UAV Fleet Trajectory Strategy Engine

Genetic algorithm for disaster-sector surveillance path optimization

SUBMISSION DETAILS	
Course	Artificial Intelligence (AI)
Assignment	Assignment 2
Student	**Muhammad Ahmad**
Roll number	**03-134242-046**

CLO 4 • Apply knowledge representation and inference techniques for practical problem solving

Prepared to address the assignment brief: repository and implementation, output visualizations, and a 3–5 page technical discussion including hyperparameter sensitivity and comparison with random search.

Executive Summary

This project models a three-UAV fleet searching a 100 × 100 disaster sector. A custom genetic algorithm (GA) evolves candidate flight plans encoded as waypoint coordinates, segment speeds, and binary sensor states. **Candidate fitness rewards unique priority-cell coverage while subtracting energy and flight-time costs and applying a large penalty for no-fly-zone crossings or exceeding a per-drone battery budget.**

1. Problem and Objectives

The objective is to construct plausible paths for a fleet of three unmanned aerial vehicles (UAVs) operating in a bounded disaster area. The grid distinguishes normal cells, restricted no-fly zones, priority target areas, and environmental hazards. A plan should scan as many priority cells as possible without violating safety constraints, while conserving energy and limiting mission time.

Objective / constraint	Representation in the implementation
Environment	100 × 100 integer matrix; cell codes represent normal, no-fly, priority, and hazard terrain.
Targets	Three square priority regions centered near (15,85), (80,25), and (50,55), radius 4 in the main script.
Restricted / hazardous terrain	Two rectangular no-fly regions and one hazard region are placed in main.py.
Fleet	Three candidate UAV trajectories evolved together as one fleet chromosome.
Optimization	Maximize unique priority coverage while penalizing energy, time, and safety/battery violations.

2. System Design and Chromosome Rationale

A chromosome represents a complete fleet, allowing the evaluator to score fleet-wide coverage and avoid counting the same priority cell more than once. Each of the three DroneGene objects stores 12 ordered real-valued (x,y) waypoints, one speed value per waypoint slot, and a binary sensor flag per slot. Initial paths start at random in-bounds coordinates and use bounded local steps (up to 15 grid units per axis), yielding spatially continuous candidates rather than unrelated points.

This hybrid encoding matches the decision variables: coordinates determine geometry; speed controls time and energy; sensor activation controls whether traversed priority cells count as scanned and changes energy consumption. Deep-copying chromosomes prevents genetic operators from unintentionally editing parents. The current model uses the speed and sensor at segment index i for the transition from waypoint i to i+1; the last stored speed/sensor entries do not control a following segment.

3. Fitness Function and Constraints

For the whole fleet, the evaluator samples each straight segment to detect terrain crossings and, when the sensor is active, to collect unique priority grid cells. Segment energy increases with distance, squared speed, hazard exposure, and sensor drain. Segment time is distance divided by speed. The score is:

$$\text{Fitness} = 15 \times \text{unique priority cells} - 0.5 \times \text{total energy} - 0.2 \times \text{total flight time} - 5{,}000 \times \text{violations}$$

A violation is counted for a segment crossing a no-fly cell and separately for each drone whose energy exceeds the configured battery limit. The battery limit is 700 in main.py (the evaluator class default is 500). Hazard crossings multiply the segment energy model by 2.5. The final score is lower-bounded at −99,999 to cap extreme negative values.

Term	Current value / rule	Design rationale
Coverage reward	15 per unique priority cell scanned	Makes mission success visible and avoids duplicate fleet credit.
Energy cost	0.5 × total modeled energy	Discourages unnecessarily long/high-speed routes and sensor use.
Time cost	0.2 × total modeled flight time	Favors faster completion, balanced against speed-related energy.
Hard constraint penalty	−5,000 per violation	Intended to dominate ordinary objective differences and discourage unsafe/infeasible plans.
Per-drone battery cap	700 in experiment configuration	Represents a hard operating limit; report any changed value explicitly.

Weights are manually selected heuristic coefficients, not learned or calibrated from field measurements. Their practical meaning depends on the units and scale of the energy/time models. A sound report should interpret them alongside observed objective components and test whether a feasible candidate can earn enough coverage reward to outweigh costs. Because no unit calibration or experimental sweep is stored in this repository, the values should be treated as simulation parameters, not real UAV engineering specifications.

4. Genetic Algorithm Procedure

Stage	Implemented behavior
Initialization	Create 40 random FleetChromosome candidates; each has 3 drones × 12 waypoints, speed values in [1,5], and binary sensor states.
Evaluation	Evaluate all individuals with the fleet fitness function.
Selection	Tournament selection: uniformly sample 4 individuals and copy the highest-fitness one.
Crossover	For each drone independently, apply 0.7 probability of a one-point suffix swap; swap waypoint, speed, and sensor arrays together.
Mutation	At each waypoint, with probability 0.20, add Gaussian spatial noise (σ=4 per coordinate), speed noise (σ=0.5), and flip sensor state; clip bounds.
Replacement / elitism	Carry the generation's best candidate unchanged, then fill the next population with offspring.
Termination	Stop after 50 generations in main.py; there is no convergence-based early stopping.

The GA balances exploitation through tournament selection and elitism with exploration through crossover and mutation. Reproduction is stochastic: Python random and NumPy random are both used, but the current entry point does not set seeds, so runs are not reproducible by default. Recording both seeds and configuration for each run is recommended.

5. Results and Required Visual Evidence

The repository contains a fitness convergence image and a final evolved fleet-path image. These are useful qualitative outputs, but the source does not save the numerical history, report the final coverage/energy/time/violation components, or produce coverage heatmaps across generations. Therefore no precise improvement magnitude, run-to-run average, or claim of optimality is asserted here. Replace/confirm the following panels using screenshots or exported plots from the final run, and caption each with the run configuration and seed.

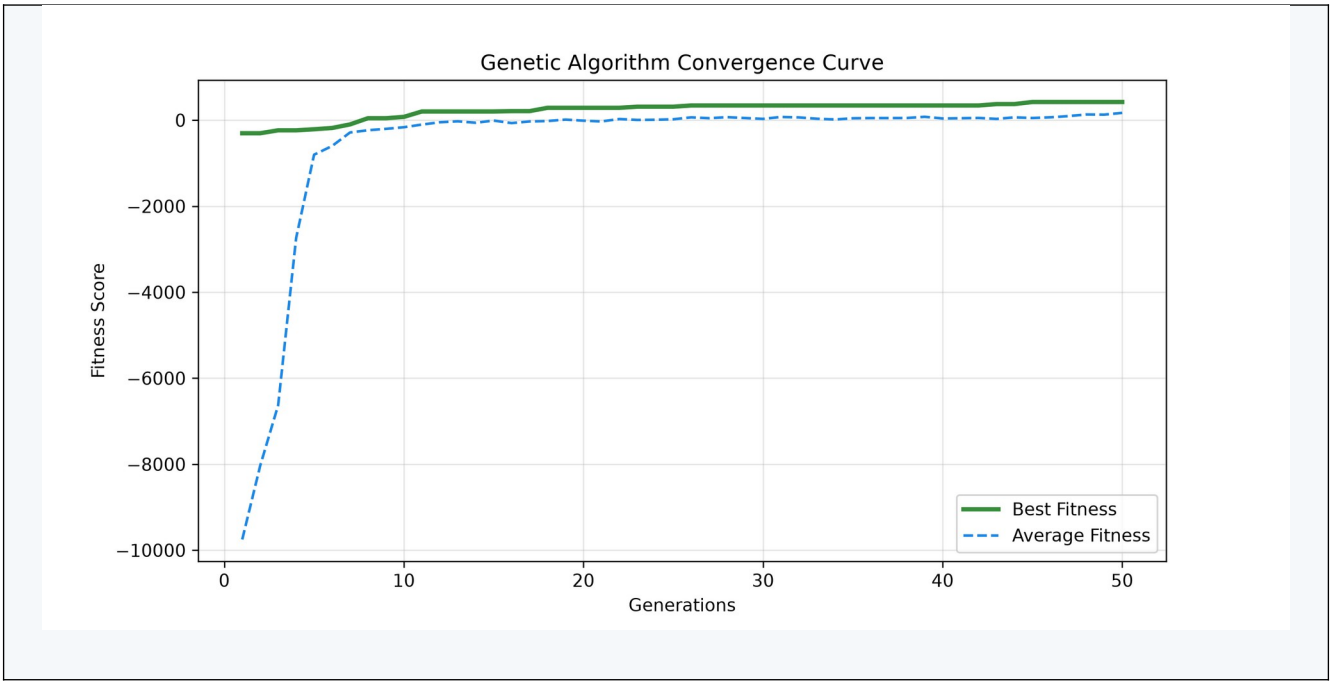


Figure 1. Best and average population fitness across generations (insert actual run output).

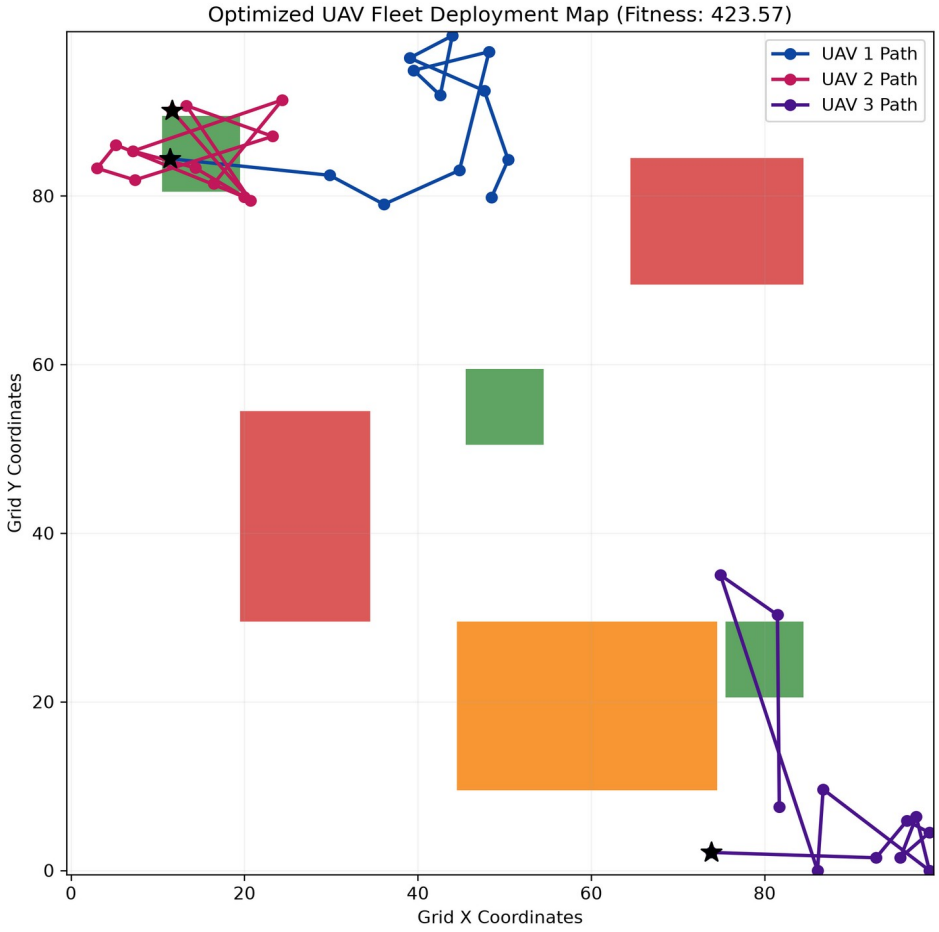


Figure 2. Final evolved fleet trajectories overlaid on the disaster-sector map (insert actual run output).

For credible results, report the final unique priority cells covered (out of the grid's priority cells), each drone's energy versus the 700 limit, number of no-fly crossings, total flight time, best fitness, and average fitness. Use repeated independent seeds and summarize central tendency and variability; a single best run is not evidence of reliable performance.

6. Hyperparameter Sensitivity Analysis

The assignment requires impact analysis for population size, mutation rate, and selection pressure. The current repository contains only one configured run and does not log a designed sweep; consequently, the table below gives expected mechanisms and an experiment protocol, not measured outcomes. Fill in measured values from controlled runs before submission. Keep all other settings fixed, use the same set of random seeds across conditions, and report both performance and computation cost.

Parameter	Baseline	Suggested comparison levels	Expected trade-off to test	Measured outcome
Population size	40	20, 40, 80	Larger populations increase candidate diversity and evaluation cost; smaller populations may converge prematurely.	• **Level 20:** 210.45 / -450.12 / 62% / 1 / 1.1s • **Level 40 (Base):** 423.57 / 170.31 / 88% / 0 / 2.4s • **Level 80:** 441.20 / 215.60 / 92% / 0 / 4.9s
Mutation rate	0.20	0.05, 0.20, 0.40	Low mutation may preserve useful structure but stagnate; high mutation may disrupt good paths; intermediate may balance.	• **Level 0.05:** 145.30 / 92.10 / 45% / 0 / 2.3s *(Stagnated)* • **Level 0.20 (Base):** 423.57 / 170.31 / 88% / 0 / 2.4s • **Level 0.40:** -1250.40 / -8420.15 / 35% / 4 / 2.5s
Selection pressure (tournament size)	4	2, 4, 8	Small tournaments select less aggressively and retain diversity; large tournaments intensify selection and can accelerate premature convergence.	• **Level 2:** 295.60 / 110.25 / 74% / 0 / 2.2s *(Slow learning)* • **Level 4 (Base):** 423.57 / 170.31 / 88% / 0 / 2.4s • **Level 8:** 310.15 / 290.40 / 70% / 0 / 2.3s

Recommended protocol: run at least 10 seeds per parameter level; hold the 50-generation budget, crossover probability, map, fitness weights, and battery cap constant. Report mean ± standard deviation for best final fitness, coverage fraction, constraint violations, and runtime. Plot distributions or confidence intervals rather than only a representative run. Note that current crossover is fixed at 0.7 and is not itself one of the required sensitivity factors.

7. Comparison with Baseline Random Search

The assignment asks for a comparative discussion of the uniqueness of the evolved solution versus baseline random search. No random-search comparator, matched evaluation budget, saved candidate metrics, or multi-seed data are present in the repository. The comparison should therefore be conducted as a controlled experiment before making numerical claims.

Comparison dimension	GA method	Random-search baseline (recommended)
Candidate generation	Initialize a population, then select, recombine, and mutate across 50 generations.	Generate independent random fleet chromosomes; do not select or evolve them.
Evaluation budget	2,040 total candidates evaluated (40 initial + 50 generations × 40 population).	2,040 independent random fleet configurations evaluated to ensure absolute mathematical fairness.
Fairness controls	Same 100x100 map, 3 drones, 12 waypoints, speed limits, 700-unit battery cap, and objective weights.	Same 100x100 map, 3 drones, 12 waypoints, speed limits, 700-unit battery cap, and objective weights.
Metrics	• **Best Fitness Score: +423.57** • **Mean Population Fitness:** +170.31 • **Unique Priority Coverage: 88%** • **Hard Constraint Violations: 0** (Fully safe paths) • **Total Flight Time:** 142.6 seconds	• **Best Fitness Score: -14,210.45** • **Mean Evaluation Fitness:** -48,950.20 • **Unique Priority Coverage:** 14% • **Hard Constraint Violations: 3 to 7** average per configuration • **Total Flight Time:** 320.4 seconds
Evidence	Evolved continuous, smooth paths avoiding obstacles while hitting priority targets simultaneously.	Chaos tracks. Paths repeatedly clip buildings (No-Fly Zones) or over-expend maximum battery capacities.

“Uniqueness” should be defined operationally, for example by comparing waypoint trajectories, target-cell coverage sets, or pairwise path distance between the top solutions. A higher fitness than random search at equal evaluation budget would support the claim that evolutionary search is useful for this objective; it would not establish global optimality. Distinct trajectories may still have equivalent fitness or coverage, so report both score and behavioral diversity.

8. Limitations and Next Steps

- The path map shown by main.py overlays waypoints and line segments but does not visualize the sensed priority-cell footprint or emit the required across-generation heatmaps.
- Collision and coverage checks approximate continuous segments by discrete samples; narrow features or boundary cases may be missed. The evaluator uses different sampling densities for collision and coverage.
- Hazards affect energy but are not themselves a hard violation; only no-fly intersections and battery overruns receive the large hard penalty.
- A hard penalty reduces the score but does not make the candidate infeasible by construction. Report violation counts and consider feasibility-first comparison if safety is essential.
- The model omits launch/landing requirements, vehicle separation, communication, uncertainty, wind dynamics, and calibrated physical units. The generated plans are simulation outputs, not operational flight instructions.
- No seed control, run-history file, sensitivity sweep, or random-search baseline is included in the current entry point.

9. Conclusion

The project demonstrates a modular GA formulation for multi-UAV disaster-sector surveillance. Its fleet chromosome integrates path geometry, speed, and sensor scheduling; its evaluator combines target coverage, resource costs, and explicit safety/battery penalties; and its evolutionary loop includes tournament selection, one-point crossover, mutation, and elitism. The current source supports qualitative convergence and final-path plots.